Princess Sumaya University for Technology

King Abdullah II Faculty of Engineering



Cloud Computing and Security

(25541)

CloudTrap: A Multi-Cloud Honeypot Deception Network

Names: *Tala Ghosheh (20220123)*

Katrin Zagha (20210858)

Major: *Network and Information Security Engineering*

Instructor: *Eng. Farah Abudabaseh*

Date:*13/05/2026*

Abstract

CloudTrap is a multi-cloud honeypot deception network deployed across Amazon Web Services (AWS) and Google Cloud Platform (GCP) to capture and analyze real-world attack behavior targeting cloud infrastructure. The system implements three realistic decoy services, designed to attract attacker interactions without exposing any real assets. All the captured events are normalized into a unified JSON schema and then forwarded to an analyzer. A real-time dashboard provides visibility into attacker profiles and the threats carried out.

honeypot → logs.jsonl → analyzer → PostgreSQL → dashboard API → browser

The deployment combines containerized service, centralized logging, PostgreSQL-based event storage, cloud-native monitoring, and automated CI/CD validation through GitHub.

During deployment, the system captured over 16,000 attack events within the first days of deployment, including unsolicited credential stuffing attempts and reconnaissance probes from external IPs. CloudTrap demonstrates that a lightweight multi-cloud honeypot architecture is achievable using modern DevSecOps practices and is capable of generating meaningful threat intelligence in a live cloud environment.

Abstract **2**
1. Introduction **3**
1.1 Background 3
1.2 Motivation 4
1.3 Objectives 4
2 Cloud Architecture **5**
2.1 Overview 5
2.3 Cloud Providers and Services Used 6
Amazon Web Services (AWS) 6
EC2 6
CloudWatch 6
IAM 6
Elastic Container Registry (ECR) 6
AWS Secrets Manager 7
Elastic IP 7
Google Cloud Platform (GCP) 7
Compute Engine 7
Cloud Logging 7
GCP Artifact Registry 7
GCP Secret Manager 7
GCP Static External IP 7
2.5 Integration Flow 8
3. Core Implementation **9**
3.1 Honeypot Services 9
3.2 Containerization and Service Orchestration 13
3.2.1 Docker Compose 13

3.2.2 Container Reisters 16
3.3 Log Forwarding 17
3.4 Analyzer Service 18
3.5 PostgreSQL Database 20
3.6 Dashboard Implementation 22
3.8 Cloud Deployment 24
IAM Role Configuration 25
Security Groups 27
3.9 Logging and Cloud Integration 29
3.10 CI/CD Pipeline 30
4. Security Analysis 32
5. Evaluation & Testing 33
5.1 Honeypot Interaction Testing 33
5.2 Functional Testing 35
5.3 Load Testing 36
5.4 Performance Testing 36
5.2 Centralized Logging Validation 39
6. Challenges & Lessons Learned 39
7. Conclusion 40
8. References 41

1. Introduction

1.1 Background

Cloud computing has become a foundational component of the modern digital infrastructure, allowing organizations to deploy scalable applications, store data, and provide online services more efficiently. However, as more systems move to public cloud platforms, cloud environments are increasingly becoming attractive targets for attackers [10] [14]. Exposed APIs, cloud storage services, and login portals are frequently targeted through reconnaissance scans, credentials stuffing attacks, and unauthorized access attempts before damage occurs.

Most cloud security solutions are primarily reactive, focusing on identifying and blocking known threats after the attacker has already reached the legitimate infrastructure. This means that the organizations usually have limited visibility into the attacker's behavior.

An alternative defensive approach could be offered through Honeypot technology. Typically positioned alongside legitimate infrastructure, Honeypots work by intentionally exposing a controlled decoy system that mimics the real services of the organization, while safely capturing attacker activities. Attackers

performing automated reconnaissance or large-scale internet scanning cannot tell the difference between legitimate and decoy services. As a result, interactions with these honeypot services can provide valuable threat intelligence by recording attack techniques, request patterns, and attempted credential usage without exposing real production assets [3], [12].

Over the past few years, most existing monitoring systems have mostly focused on a single cloud provider, which provides fragmented visibility across environments. This creates difficulties in correlating suspicious activities targeting services that are distributed across multiple cloud platforms. A significant gap, therefore, exists in accessible multi-cloud deception systems capable of safely capturing and analyzing attacker behavior across distributed cloud environments [3].

1.2 Motivation

The primary motivation behind CloudTrap was to design and implement a practical multi-cloud cybersecurity platform that combines cloud deployment, deception technology, centralized logging, and threat analysis within a unified architecture. The project was developed to explore how attackers interact with exposed cloud services while simultaneously applying secure cloud engineering practices in a realistic deployment environment [2], [12].

Another motivation was to gain practical experience with modern cloud technologies including Docker containerization, cloud-native monitoring systems, CI/CD automation, and secure infrastructure deployment across multiple cloud service providers. The project also aimed to demonstrate how lightweight honeypot systems can provide valuable threat intelligence without exposing any real organizational assets or sensitive information.

1.3 Objectives

- Design and deploy a multi-cloud honeypot architecture using AWS and GCP.
- Implement realistic deception services, including a fake REST API, a fake S3 service, and a fake login portal.
- Capture and centralize attacker interaction logs using cloud-native logging solutions.
- Develop an analyzer capable of detecting suspicious behavior patterns such as credential attacks, reconnaissance attempts, and API enumeration.
- Visualize attack activity through a centralized monitoring dashboard.
- Integrate CI/CD automation for container building, validation, and deployment.
- Apply secure cloud deployment practices, including IAM-based access control, secret management, and container isolation.

2 Cloud Architecture

2.1 Overview

CloudTrap is a multi-cloud honeypot framework deployed across both Amazon Web Services (AWS) and Google Cloud Platform (GCP) environments. The system provides centralized logging, combining

deception services, automates attack analysis, and dashboard visualization, all within a unified cloud security architecture.
There are four primary layers: the deception layer, the logging layer, the analysis layer, and the presentation layer. Each layer in the architecture is responsible for a specific part of the monitoring and analysis workflow.

The first layer is the deception layer; multiple fake cloud services are designed to imitate realistic infrastructure. These services include a fake REST API; a fake web backend service that accepts HTTP requests and logs attacker interactions. A fake S3-compatible service, and a fake login portal. These services are meant to attract and safely record suspicious interactions of any attacker or automated scanning attempts[2], [12].
The logging layer is responsible for collecting interaction logs that are generated by the honeypot services. Structured JSON logs are sent to the cloud-native monitoring services, which are AWS CloudWatch and GCP Cloud Logging. These logs contain the required metadata such as timestamps, source IP address, request methods, payloads, and requested endpoint [5].
[5]analyzer service. The analyzer aggregates events, identifies suspicious behavior patterns, and classifies attacks into credential attempts, reconnaissance scans, or API enumeration activity. Processed events are stored in PostgreSQL database, which allows the dashboard to retrieve statistics, summaries, and geolocation data without relying only on raw log files [9], [11].

Finally, the presentation layer, which provides a centralized dashboard interface that is used to visualize the attack activity and summarize the threat information. The dashboard displays attacker statistics, request summaries, and categorized attack events generated by the analyzer services. As well as that, IP geolocation is also available, where the attacker's IP address is identified, in addition to their location set on a map. Filtering of service providers also exists, allowing for the separation of AWS and GCP-generated events. Before accessing the dashboard, login is first requested, which allows either an admin or an analyst to access the dashboard.

All services within CloudTrap are containerized using Docker and managed through Docker Compose to simplify deployment, scalability, and service orchestration across cloud environments [7],[8].

2.3 Cloud Providers and Services Used

Amazon Web Services (AWS)

To support the deployment and the monitoring of the CloudTrap infrastructure, six services from AWS were used.

EC2

Amazon EC2 was used as the host of the Dockerized honeypot services, which included the fake REST API and the fake S3 service. The reason EC2 was chosen as opposed to alternative services, like Amazon Lambda, is that it offers long-running services that continuously listen for incoming attacker interaction. It provides flexible virtual machine management, supports all the exposed network ports that were required to deploy a realistic honeypot, and also integrates well with Docker-based work [7].

CloudWatch

Amazon CloudWatch was used for the centralized log collection and monitoring of the AWS-hosted services. Any JSON logs generated by the honeypot containers were forwarded to the CloudWatch Logs, aggregating the attacker activities and allowing for real-time monitoring. It was chosen because it provided a scalable cloud-native logging and integrated directly with the AWS infrastructure. CloudWatch alarms were also configured to detect repeated credential attempts exceeding predefined thresholds [5].

IAM

AWS Identity and Access Management (IAM) was used to enforce role-based permissions between system components. IAM policies were set up according to the principle of least privilege to ensure the services were receiving only the minimum requirements they needed to operate [4],[6].

Elastic Container Registry (ECR)

To store Docker images generated through the CI/CD pipeline, Amazon ECR was used. Docker packages each service with its dependencies and runtime environment, while ECR acts as the private registry that stores all these images. This allows cloud instances to pull validated container images directly from a centralized registry during deployment [7].

AWS Secrets Manager

To securely store any sensitive credentials required by CloudTrap, including dashboard login credentials and database-related secrets. This includes the dashboard login credentials and database-related secrets.

Elastic IP

An Elastic IP address was assigned to the AWS-hosted infrastructure to provide a stable public reach point for the services and the dashboard. Using a fixed public IP simplifies external access, testing, and monitoring. (the instance Elastic IP is 13.63.224.134)

Google Cloud Platform (GCP)

To support the multi-cloud deployment, several Google Cloud services were also used.

Compute Engine

A Google Compute Engine instance was provisioned to host the fake login portal deployed by the GCP side of CloudTrap. Similar to the AWS EC2 deployment, the service was containerized using Docker to maintain deployment consistency across all the environments.

Cloud Logging

Google Cloud Logging was used to collect logs that are generated by the GCP-hosted login page. These logs were integrated along with the AWS ones to support the centralized monitoring and analysis. This way, the analyzer and dashboard components can process events originating from both cloud providers.

GCP Artifact Registry

To store the Docker image for the GCP-hosted login page. Similar to the Amazon ECR, it provides a centralized location for storing container images before deployment. This allows the login portal image to be managed and deployed consistently within the GCP environment.

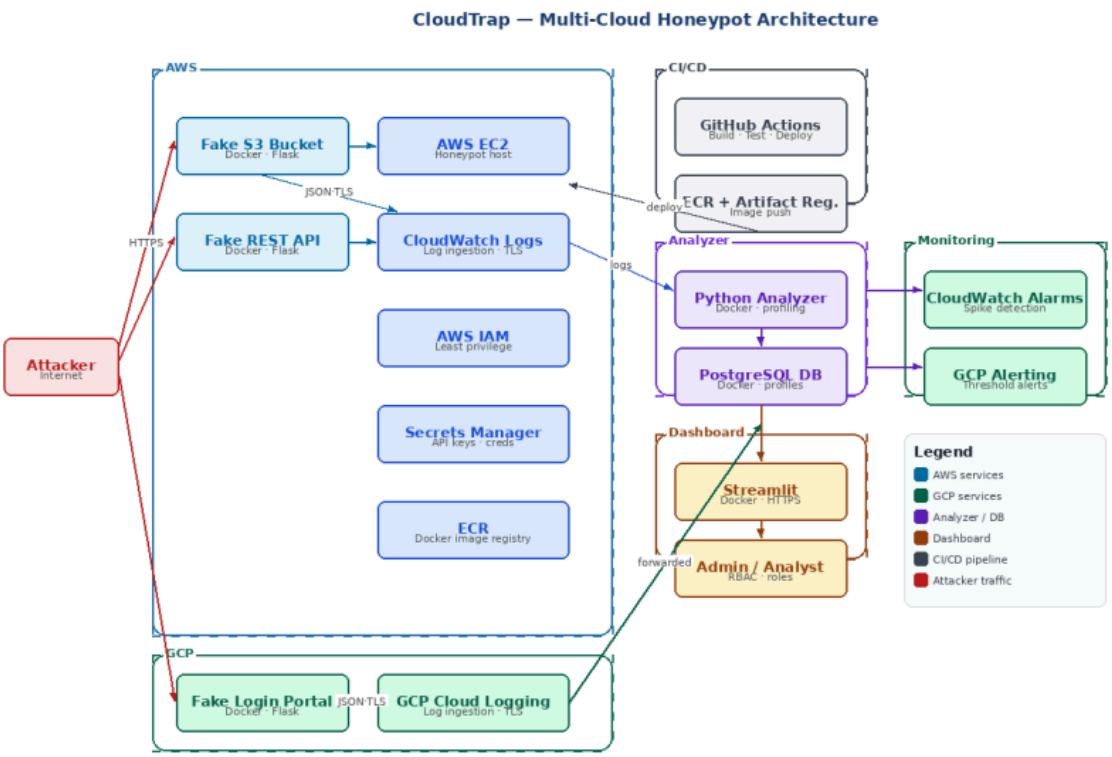
GCP Secret Manager

GCP Secret Manager was initially considered for managing deployment secrets on the GCP side. However, the final implementation relied primarily on AWS Secrets Manager because the centralized dashboard and database services were hosted within the AWS deployment environment. [4],[6]

GCP Static External IP

A static external IP address was reserved for the GCP Compute Engine instance hosting the login portal. This provided a stable public endpoint for testing, attacker interaction capture, and firewall rule consistency. (136.112.90.13) [7]

2.4 Diagram



2.5 Integration Flow

The workflow begins when an attacker interacts with one of the publicly exposed honeypot services. Each service records the request using a structured JSON logging format containing metadata such as timestamps, source IP address, cloud provider, request path, HTTP method, headers, payload information, and user-agent details.

Within AWS, these events are forwarded into Amazon CloudWatch Logs. On the GCP deployment, events generated by the login portal are forwarded to Google Cloud Logging.

For the GCP-hosted login portal, attacker interactions are stored in a local JSONL log file and are also forwarded into Google Cloud Logging. A dedicated GCP forwarder service continuously monitors the login portal log file and detects newly generated events. These events are then sent to an AWS-hosted ingest endpoint through an authenticated HTTP request.

On the AWS side, a lightweight Flask-based receiver service accepts forwarded login events through the /ingest endpoint. The receiver validates an authentication token and appends accepted events into the shared login-portal JSONL log file used by the analyzer.

The analyzer continuously processes logs generated from both AWS and GCP services, classifies suspicious behavior, enriches source IP addresses using geolocation lookups, and stores processed events inside PostgreSQL. The dashboard retrieves this information directly from PostgreSQL and visualizes attack statistics, attacker profiles, severity levels, and geolocation information.[5],[11]

3. Core Implementation

3.1 Honeypot Services

Three lightweight containerized services were implemented, designed to imitate commonly exposed cloud assets. These services were Flask-based and consisted of a fake REST API, a fake S3 storage endpoint, and a fake login portal. They were designed to appear realistic enough to attract automated scanners without exposing any real data. All these services were implemented using Flask and deployed inside a Docker container.

The **fake REST API** runs on port 5000 and simulates a generic cloud backend service. It accepts common HTTP requests such as GET and POST requests, including ones sent to sensitive paths like /admin. Instead of implementing the request as is, the service records the required metadata to log the event and returns a generic error response intended to resemble restricted or unavailable API endpoints.[2]

```
142
143     @app.route(
144         "/",
145         defaults={"path": ""},
146         methods=["GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS", "HEAD"],
147     )
148     @app.route(
149         "/<path:path>",
150         methods=["GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS", "HEAD"],
151     )
```

Incoming requests are processed through a catch-all route that captures the required metadata. Each interaction is then classified into a simplified attacker action category, such as reconnaissance attempts, credential attempts, and payload submission. Logs are structurally stored in JSONL format and then forwarded to AWS CloudWatch logs to support the centralized monitoring and for later analysis.[5],[9]

```python
def catch_all(path):
    full_path = "/" + path

    log_entry = {
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "service": "api-service",
        "cloud_provider": os.getenv("CLOUD_PROVIDER", "aws"),
        "source_ip": request.headers.get("X-Forwarded-For", request.remote_addr),
        "method": request.method,
        "path": full_path,
        "query_string": request.query_string.decode("utf-8", errors="replace") or None,
        "user_agent": request.headers.get("User-Agent"),
        "headers": filtered_headers(),
        "payload": safe_payload(),
        "action": classify_action(full_path, request.method),
    }

    write_log(log_entry)
    return fake_response(full_path)

```

The service intentionally avoids revealing any sensitive backend information and always returns controlled failure responses such as HTTP 401,403, or 503 codes just like a real production would behave.

```python
def fake_response(path: str):
    path_lower = path.lower()

    if "login" in path_lower or "auth" in path_lower:
        return jsonify({"error": "Invalid credentials"}), 401

    if "admin" in path_lower:
        return jsonify({"error": "Forbidden"}), 403

    if path_lower.startswith("/api"):
        return jsonify({"error": "Unauthorized", "code": 401}), 401

    return jsonify({"message": "Service temporarily unavailable"}), 503

```

The **fake S3 service** runs on port 5001, and it imitates a cloud object storage endpoint. It captures attempts related to bucket enumeration, object access, and upload behavior. This is helpful because exposed storage services are common targets for attackers to try and discover publicly accessible files or misconfigured buckets[10]. Incoming requests are processed through dedicated routes that capture metadata, including timestamps, source IP addresses, HTTP methods, requested bucket names, object keys, request headers, query strings, and submitted payloads. Just like the REST API service, each event is stored in structured JSONL format and simultaneously forwarded into AWS CloudWatch Logs.

```
def s3_error_xml(code: str, message: str, status: int) -> Response:
    body = (
        '<?xml version="1.0" encoding="UTF-8"?>'
        "<Error>"
        f"<Code>{code}</Code>"
        f"<Message>{message}</Message>"
        "<RequestId>CLOUDTRAP000000000001</RequestId>"
        "</Error>"
    )
    return Response(body, status=status, mimetype="application/xml")
```

To improve realism, the honeypot exposes predefined fake bucket names such as internal-assets and customer-exports while consistently returning controlled “Access Denied” responses for unauthorized object operations. This imitates the behavior of restricted cloud storage environments while safely capturing attacker interaction patterns without exposing any real files or sensitive data.

```
35  FAKE_BUCKETS = [
36          "prod-backups-2024",
37          "internal-assets",
38          "customer-exports",
39      ]
40
```

The **fake login** runs on port 5002, the service was hosted on a GCP Compute Engine instance, and it imitates an enterprise authentication interface. Any submitted credentials are treated as attacker-supplied input and are logged. The service records attacker interactions, including page visits, credential submission attempts, and generic path enumeration.

Again, just like other services, the events are logged in a structured way, while events are forwarded into Google Cloud Logging to support centralized multi-cloud monitoring and later attack analysis [5]. Logging behavior is controlled using environment-based configuration variables, allowing the service to adapt dynamically depending on the deployment environment. The service always returns a generic authentication failure response.

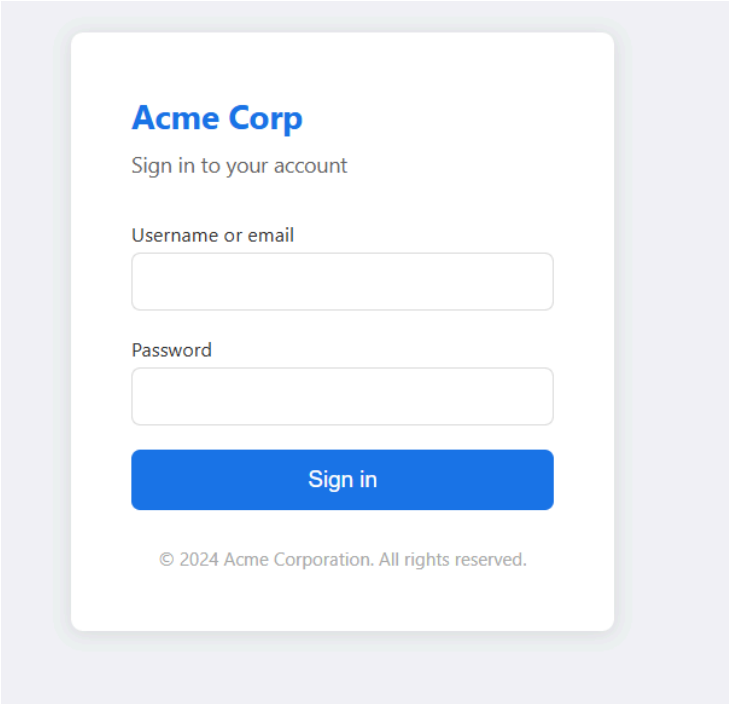
```python
# AWS CLOUDWATCH CONFIG
CLOUDWATCH_ENABLED = os.getenv("CLOUDWATCH_ENABLED", "false").lower() == "true"
AWS_REGION = os.getenv("AWS_REGION", "eu-north-1")
LOG_GROUP = os.getenv("CLOUDWATCH_LOG_GROUP", "cloudtrap-logs")
LOG_STREAM = os.getenv("CLOUDWATCH_LOG_STREAM", "login-portal")

logs_client = boto3.client("logs", region_name=AWS_REGION) if CLOUDWATCH_ENABLED else None

# GCP CLOUD LOGGING CONFIG
GCP_LOGGING_ENABLED = os.getenv("GCP_LOGGING_ENABLED", "false").lower() == "true"
GCP_LOG_NAME = os.getenv("GCP_LOG_NAME", "cloudtrap-login-portal")

gcp_logger = None

if GCP_LOGGING_ENABLED:
    if gcp_logging is None:
        print("[GCP Logging setup error] google-cloud-logging is not installed", flush=True)
    else:
        try:
            gcp_client = gcp_logging.Client()
            gcp_logger = gcp_client.logger(GCP_LOG_NAME)
            print("[+] GCP Cloud Logging enabled", flush=True)
        except Exception as e:
            print(f"[GCP Logging setup error] {e}", flush=True)
```



```python

def send_to_cloudwatch(entry: dict):
    if not CLOUDWATCH_ENABLED:
        return

    try:
        logs_client.put_log_events(
            logGroupName=LOG_GROUP,
            logStreamName=LOG_STREAM,
            logEvents=[{
                "timestamp": int(time.time() * 1000),
                "message": json.dumps(entry, ensure_ascii=False),
            }],
        )
    except Exception as e:
        print(f"[CloudWatch error] {e}", flush=True)


def send_to_gcp_logging(entry: dict):
    if not GCP_LOGGING_ENABLED or gcp_logger is None:
        return

    try:
        gcp_logger.log_struct(entry, severity="INFO")
    except Exception as e:
        print(f"[GCP Logging error] {e}", flush=True)

```

+

3.2 Containerization and Service Orchestration

3.2.1 Docker Compose

All CloudTrap components were containerized using Docker to ensure consistent deployment across cloud environments. Containerization simplified dependency management because each service runs within its own isolated Python environment, together with the required libraries and runtime dependencies. This improved portability and reproducibility, allowing the same services to be deployed consistently across different environments without manual package installation or configuration.[7]

During development, the original docker-compose.yml file was used as the base orchestration configuration for local testing and integration. For the final multi-cloud deployment, the configuration was separated into two cloud-specific Docker Compose files: docker-compose.aws.yml on the AWS EC2 instance and docker-compose.gcp.yml on the GCP Compute Engine instance. This separation allowed each cloud environment to maintain its own services, environment variables, exposed ports, and deployment-specific settings while still supporting centralized integration between both providers.

The AWS deployment file orchestrated the centralized CloudTrap environment, including the API honeypot, fake S3 service, analyzer, PostgreSQL database, dashboard, AWS ingest receiver, and

supporting monitoring services. The GCP deployment file was responsible for orchestrating the fake login portal together with the GCP forwarding service, which securely forwarded login events to the centralized AWS environment.

The Docker Compose configurations defined service dependencies, restart policies, exposed ports, environment variables, internal networking, persistent storage volumes, and container health checks. Persistent Docker volumes were configured to preserve PostgreSQL data, shared JSONL log files, and analyzer reports across container restarts.

Overall, Docker Compose significantly improved deployment consistency and simplified orchestration by allowing both AWS and GCP environments to be recreated using version-controlled infrastructure configurations.

The original docker-compose.yml:

```yaml
api-service:
  build:
    context: ./services/api-service
    dockerfile: Dockerfile
  container_name: cloudtrap-api
  ports:
    - "5000:5000"
  volumes:
    - api-logs:/app/logs
  environment:
    - CLOUD_PROVIDER=aws
    - LOG_FILE=/app/logs/logs.jsonl
    - CLOUDWATCH_ENABLED=true
    - AWS_REGION=eu-north-1
    - CLOUDWATCH_LOG_GROUP=cloudtrap-logs
    - CLOUDWATCH_LOG_STREAM=api-service
  restart: always
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:5000/_internal/health"]
    interval: 30s
    timeout: 5s
    retries: 3
```

docker-compose.aws.yml :

```yaml
  api-service:
    build:
      context: ./services/api-service
      dockerfile: Dockerfile
    container_name: cloudtrap-api
    ports:
      - "5000:5000"
    volumes:
      - api-logs:/app/logs
    environment:
      CLOUD_PROVIDER: aws
      LOG_FILE: /app/logs/logs.jsonl
      CLOUDWATCH_ENABLED: "true"
      AWS_REGION: eu-north-1
      CLOUDWATCH_LOG_GROUP: cloudtrap-logs
      CLOUDWATCH_LOG_STREAM: api-service
      PYTHONUNBUFFERED: "1"
    restart: always
    healthcheck:
```

Docker-compose.gcp.yml:

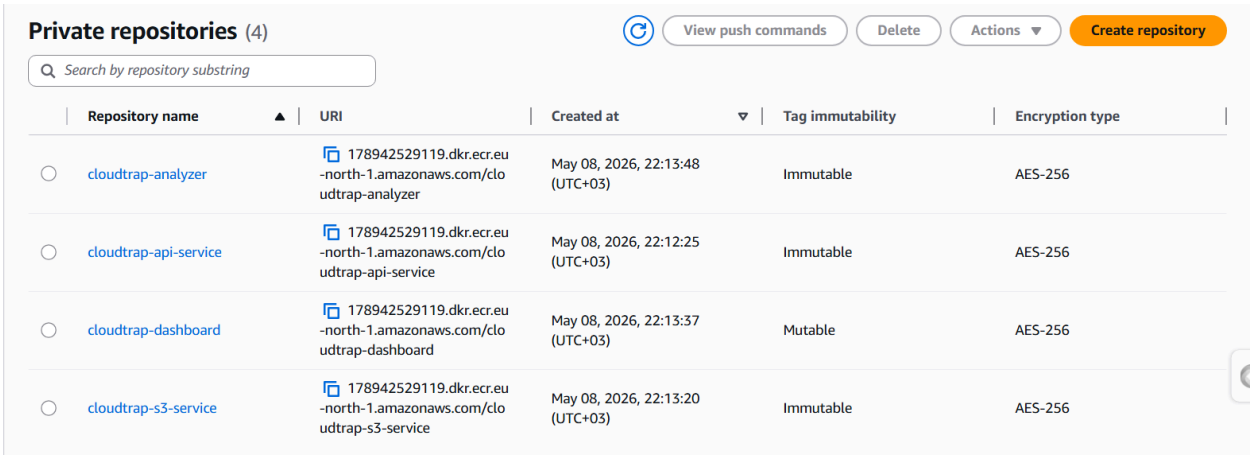
```yaml
services:

  login-portal:
    build:
      context: ./services/login-portal
      dockerfile: Dockerfile
    container_name: cloudtrap-login
    ports:
      - "5002:5002"
    volumes:
      - login-logs:/app/logs
    environment:
      CLOUD_PROVIDER: gcp
      LOG_FILE: /app/logs/logs.jsonl
      CLOUDWATCH_ENABLED: "false"
      GCP_LOGGING_ENABLED: "true"
      GCP_LOG_NAME: cloudtrap-login-portal
      PYTHONUNBUFFERED: "1"
    restart: always
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5002/_internal/health"]
      interval: 30s
      timeout: 5s
      retries: 3
```

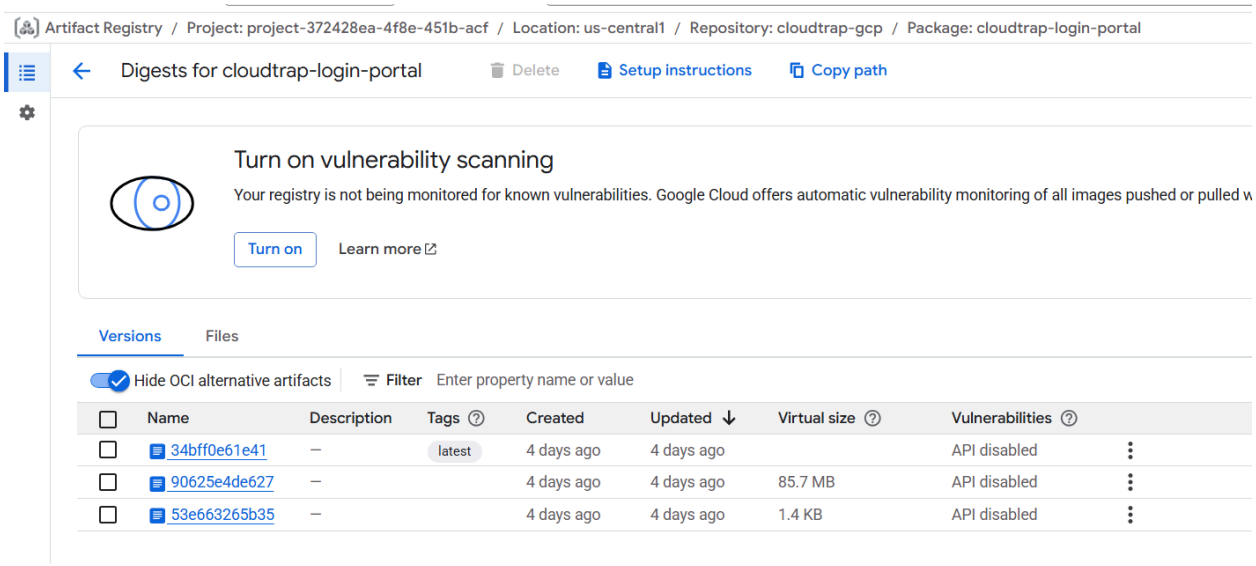
```
ubuntu@ip-172-31-39-55:~/CloudTrap$ docker compose ps
WARN[0000] /home/ubuntu/CloudTrap/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please re
move it to avoid potential confusion
NAME                    IMAGE                     COMMAND                  SERVICE         CREATED          STATUS
            PORTS
cloudtrap-analyzer      cloudtrap-analyzer        "sh -c 'while true; …"   analyzer        11 hours ago     Up 6 minut
es
cloudtrap-api           cloudtrap-api-service     "python app.py"          api-service     51 minutes ago   Up 6 minut
es (healthy)   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
cloudtrap-dashboard     cloudtrap-dashboard       "python app.py"          dashboard       2 hours ago      Up 6 minut
es (healthy)   0.0.0.0:5003->5003/tcp, [::]:5003->5003/tcp, 0.0.0.0:443->5003/tcp, [::]:443->5003/tcp
cloudtrap-log-forwarder   cloudtrap-log-forwarder   "python forwarder.py"    log-forwarder   14 hours ago     Up 6 minut
es
cloudtrap-login         cloudtrap-login-portal    "python app.py"          login-portal    10 hours ago     Up 6 minut
es (healthy)   0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp
cloudtrap-postgres      postgres:15-alpine        "docker-entrypoint.s…"   postgres        4 days ago       Up 6 minut
es (healthy)   5432/tcp
cloudtrap-s3            cloudtrap-s3-service      "python app.py"          s3-service      14 hours ago     Up 6 minut
es (healthy)   0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
ubuntu@ip-172-31-39-55:~/CloudTrap$
```

3.2.2 Container Reisters

Docker images generated through CI/CD process were stored inside centralized registries before deployment. AWS-hosted service images were managed through Amazon ECR, the container registries were configured as private repositories to prevent unauthorized public access to deployment images. Most repositories also use immutable image tags, ensuring that previously published container versions cannot be overwritten.
This improved deployment consistency, traceability, and CI/CD reliability across the multi-cloud environment.[7], [8]



As for the GCP-hosted login page container image was stored in the Google Artifact Registry, providing centralized image storage and version management. Multiple image versions were maintained to support consistent redeployment and controlled testing.



3.3 Log Forwarding

A cross-cloud log forwarding mechanism was implemented to connect the GCP-hosted login portal with the centralized AWS analysis environment. This was necessary because the login portal was deployed separately on Google Cloud Platform, while the analyzer, PostgreSQL database, and dashboard were hosted on AWS.

The GCP login portal records attacker interactions in a local JSONL log file. These events are also forwarded into Google Cloud Logging for native monitoring within the GCP environment. To make the same events available to the centralized analyzer, a dedicated GCP forwarder service was implemented.

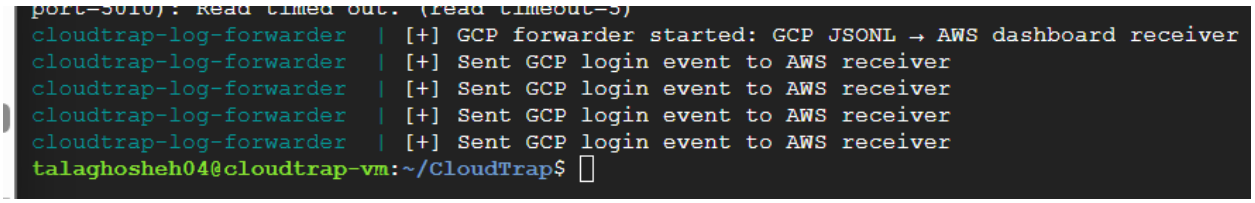
The GCP forwarder continuously monitors the login portal JSONL file and keeps track of the last file position it has read. This prevents the same event from being forwarded repeatedly. When new log lines are detected, the forwarder parses each line as a JSON event and sends it to the AWS central ingest receiver using an HTTP POST request.

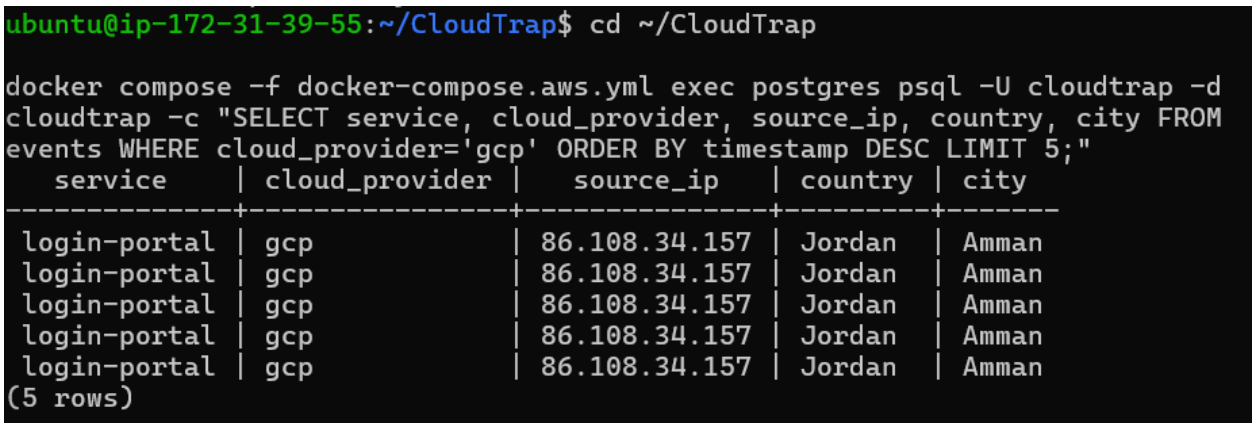
The forwarding request includes an authentication header called X-CloudTrap-Token. This token is used to verify that the event was sent by the trusted CloudTrap forwarding service and not by an unauthorized external source.

On the AWS side, a lightweight Flask-based receiver service was implemented. The receiver exposes an /ingest endpoint on port 5010. When a forwarded event reaches this endpoint, the receiver checks the X-CloudTrap-Token header. If the token is invalid, the request is rejected with an unauthorized response. If the token is valid, the receiver appends the event to the shared login-portal JSONL log file used by the analyzer.

This design allowed GCP-generated login events to remain visible in Google Cloud Logging while also being centralized into the AWS analysis pipeline. As a result, the analyzer and dashboard could process AWS and GCP attacker events together while still preserving the original cloud-provider field for filtering and investigation.

To validate successful forwarding, PostgreSQL queries were executed on the AWS deployment environment to confirm that GCP-generated login events were successfully received, analyzed, enriched, and stored inside the centralized database. The forwarded events remained identifiable using the cloud_provider='gcp' field, allowing separation between AWS and GCP attacker activity during investigation[5].





3.4 Analyzer Service

The analyzer is a Python-based service responsible for processing raw honeypot logs and transforming them into meaningful security intelligence. It reads structured log entries, classifies events, assigns severity levels, enriches source IP addresses with geolocation data, and stores processed results in PostgreSQL.[9] [11]

The analyzer classifies attacker behavior into several categories, including credential attempts, API reconnaissance, bucket enumeration, object access attempts, and object upload attempts. These categories are based on request paths, HTTP methods, submitted payloads, and service-specific behavior[9].

Severity scoring was added to help prioritize events, ranging from low to critical depending on the detected behavior. For example, repeated login attempts or access to sensitive endpoints are treated as more suspicious than ordinary low-risk requests.

```
def classify_event(log):
    service = log.get("service", "")
    path = log.get("path", "").lower()
    method = log.get("method", "").upper()
    action = log.get("action", "").lower()
    payload = str(log.get("payload", "")).lower()
    command = log.get("command", "").lower()
    attempts = log.get("attempts", 0)

    if service == "api-service":
        if "login" in path or "auth" in path or "password" in payload:
            attack = "credential_attempt"
        elif method == "GET":
            attack = "api_reconnaissance"
        elif method in ("POST", "PUT", "PATCH", "DELETE"):
            attack = "api_abuse_attempt"
        else:
            attack = "unknown"
```

The analyzer also groups events by source IP address, allowing the system to build attacker profiles that include total event count, attack types, severity distribution, and timeline of activity. Geolocation enrichment was also done through external IP lookup services, which allows the dashboard to visualize attacker information such as country, city, ISP, and approximate the geographic coordinates.

```python
def get_ip_geolocation(ip):
    """Lookup geolocation for an IP using ip-api.com."""

    if not ip or ip in ("unknown", "127.0.0.1", "::1", "localhost"):
        return {
            "country": "Unknown",
            "city": "Unknown",
            "isp": "Unknown",
            "latitude": None,
            "longitude": None,
        }

    try:
        response = requests.get(f"http://ip-api.com/json/{ip}", timeout=5)
        data = response.json()

        if data.get("status") == "success":
            return {
                "country": data.get("country", "Unknown"),
                "city": data.get("city", "Unknown"),
                "isp": data.get("isp", "Unknown"),
                "latitude": data.get("lat"),
                "longitude": data.get("lon"),
            }
```

3.5 PostgreSQL Database

PostgreSQL was used as the central database for processed attack events. It stores analyzed events, attacker profiles, severity values, cloud provider tags, timestamps, and geolocation fields. PostgreSQL was chosen because it supports concurrent access, allowing the analyzer to continuously write new events while the dashboard reads updated statistics.

This is more reliable than relying only on raw log files, especially as the number of captured events increases. In the final implementation, the system successfully handled tens of thousands of events while keeping the dashboard responsive.

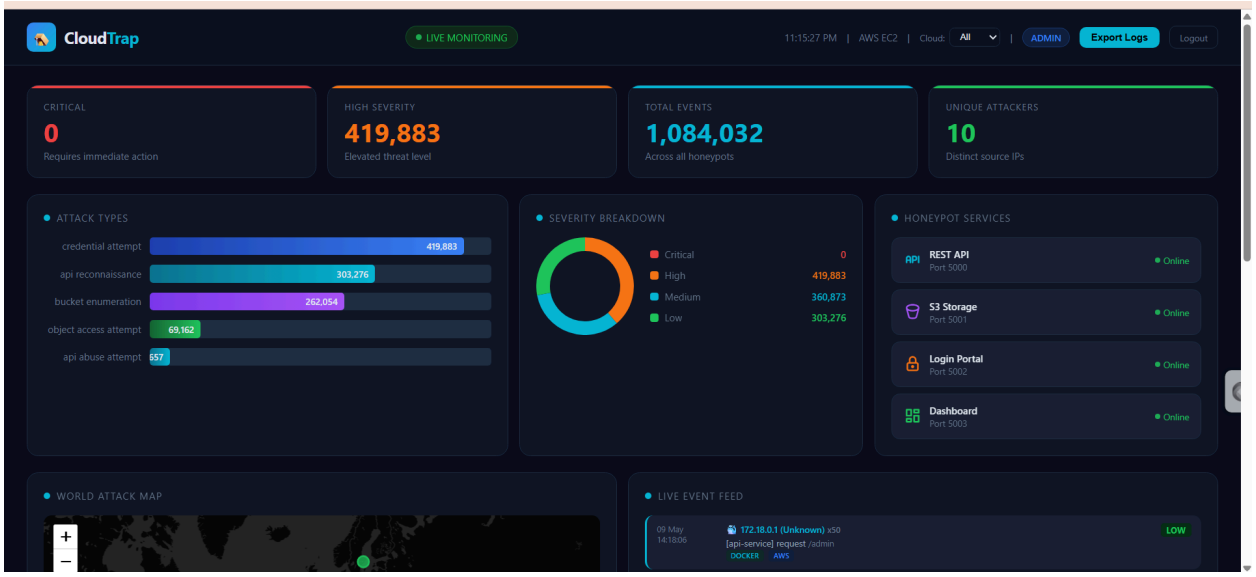
Two main databases were implemented. The events table stores processed attack events with their metadata. The attacker_profile table aggregates activity according to the source IP address and maintains summarized attacker intelligence.

```sql
CREATE TABLE IF NOT EXISTS events (
    id              SERIAL PRIMARY KEY,
    timestamp       TIMESTAMPTZ,
    service         TEXT,
    source_ip       TEXT,
    method          TEXT,
    path            TEXT,
    action          TEXT,
    command         TEXT,
    attack_type     TEXT,
    severity        TEXT,
    cloud_provider  TEXT,
    country         TEXT,
    city            TEXT,
    isp             TEXT,
    latitude        FLOAT,
    longitude       FLOAT,
    raw             JSONB,
    inserted_at     TIMESTAMPTZ DEFAULT NOW()
);
```

```sql
CREATE TABLE IF NOT EXISTS attacker_profiles (
    source_ip     TEXT PRIMARY KEY,
    event_count   INT DEFAULT 0,
    threat_score  INT DEFAULT 0,
    top_severity  TEXT,
    attack_types  TEXT[],
    first_seen    TIMESTAMPTZ,
    last_seen     TIMESTAMPTZ,
    updated_at    TIMESTAMPTZ DEFAULT NOW()
);
```

3.6 Dashboard Implementation

The dashboard provides the presentation layer of CloudTrap. It runs on port 5003 and gives users a centralized view of attacker activity across the honeypot environment. The dashboard includes real-time statistic cards, attack type charts, severity distribution charts, a geolocation map, attacker profiles, live event logs, and cloud provider filtering.



The backend was implemented using Flask and communicates directly with the PostgreSQL database in order to retrieve the processed information generated by the analyzer. The dashboard backend exposes several API endpoints, including /api/report for summary statistics, /api/events for paginated event logs, /api/attack-map for geolocation data, /api/attacker/<ip> for individual attacker profiles, and /api/stream for real-time event updates. Filtering functionality allows events to be separated according to severity level, service type, source IP address, and cloud provider origin, supporting more effective investigation and attack analysis workflows.

```python
@app.route("/api/report")
def api_report():
    role = check_auth()
    if not role:
        return jsonify({"error": "Unauthorized"}), 401

    severity_filter = request_arg("severity")
    service_filter = request_arg("service")
    cloud_filter = request_arg("cloud_provider")
    ip_filter = request_arg("ip")

    conn = None

    try:
        conn = get_db()
        cur = conn.cursor()

        conditions = []
        params = []
```

```python

@app.route("/api/events")
def api_events():
    role = check_auth()
    if not role:
        return jsonify({"error": "Unauthorized"}), 401

    page = max(1, int(request_arg("page") or 1))
    limit = min(100, int(request_arg("limit") or 50))
    offset = (page - 1) * limit

    conn = None

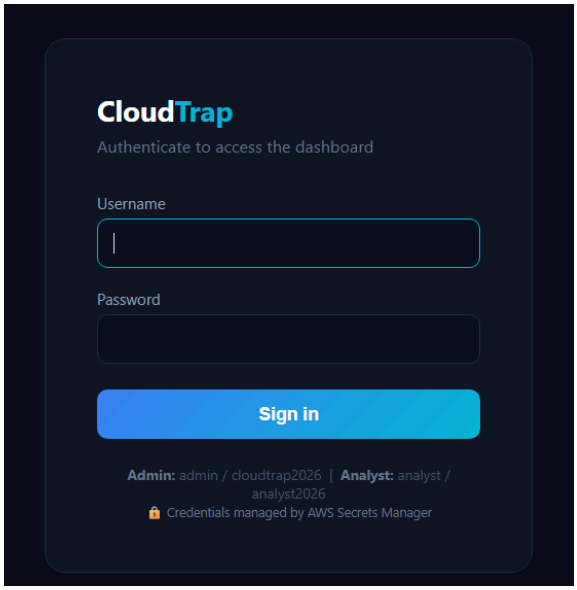
```

Role-based access control was also implemented. The admin account has full access, including the ability to export logs, while the analyst account has read-only access. Authentication tokens are stored through secure HTTP-only cookies, while the credentials and database secret are retrieved dynamically from AWS Secrets Manager instead of being hardcoded. This supports the security requirement that sensitive attacker intelligence should not be publicly accessible [4] [6]. Note: The credentials were only displayed on the login page to simplify testing; in a real deployment, they will be kept securely hidden.

```python

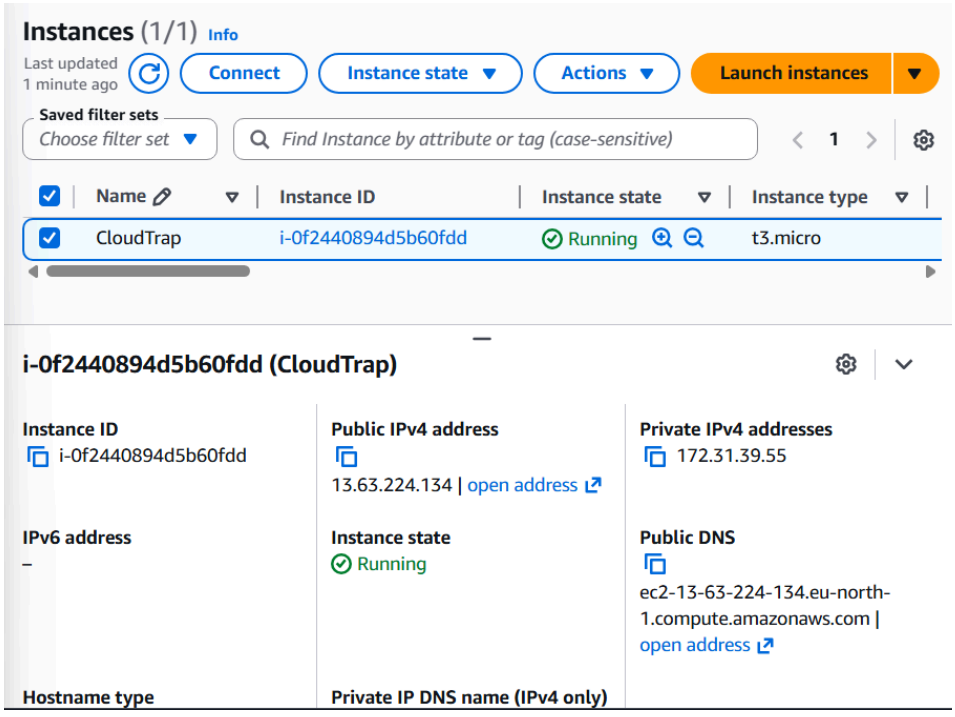
def fetch_secrets():
    try:
        client = boto3.client("secretsmanager", region_name=AWS_REGION_SM)
        response = client.get_secret_value(SecretId=SECRET_ARN)
        print("[+] Secrets loaded from AWS Secrets Manager")
        return json.loads(response["SecretString"])
    except Exception as e:
        print(f"[!] Secrets Manager unavailable ({e}), using env fallback")
        return {}

_secrets = fetch_secrets()
ADMIN_PASS = _secrets["ADMIN_PASS"]
ANALYST_PASS = _secrets["ANALYST_PASS"]
POSTGRES_PASSWORD = _secrets["POSTGRES_PASSWORD"]
```



3.8 Cloud Deployment

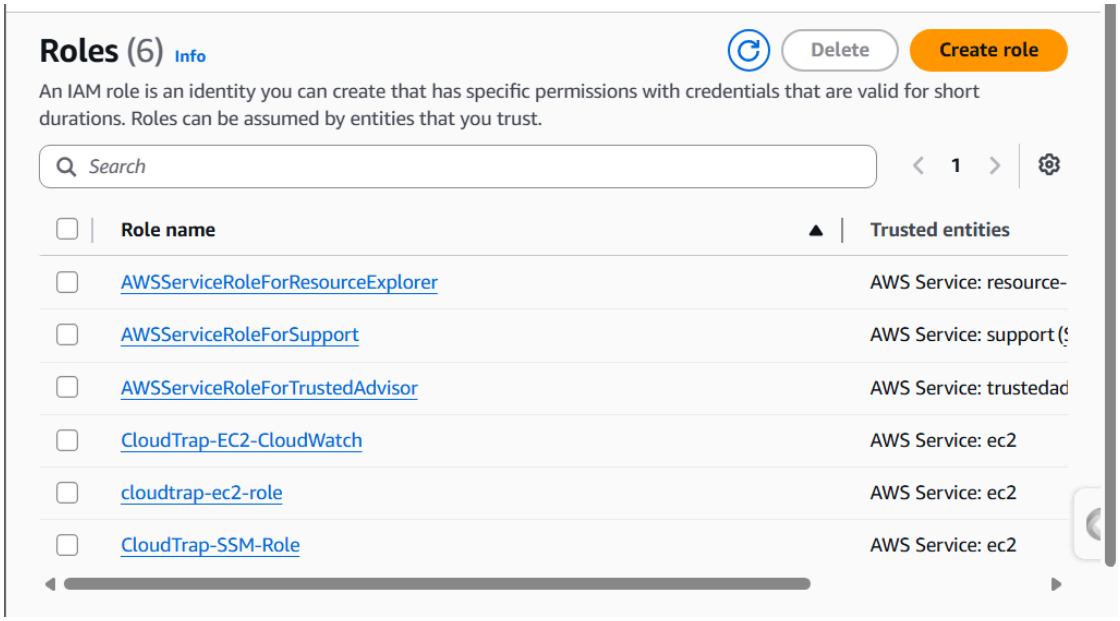
On AWS, CloudTrap was deployed on an **EC2** instance running Ubuntu.



Docker and Docker Compose were installed on the instance, and the full containerized stack was deployed. The AWS deployment includes the API honeypot, S3 honeypot, login portal, analyzer, dashboard, PostgreSQL database, and log-forwarder service.

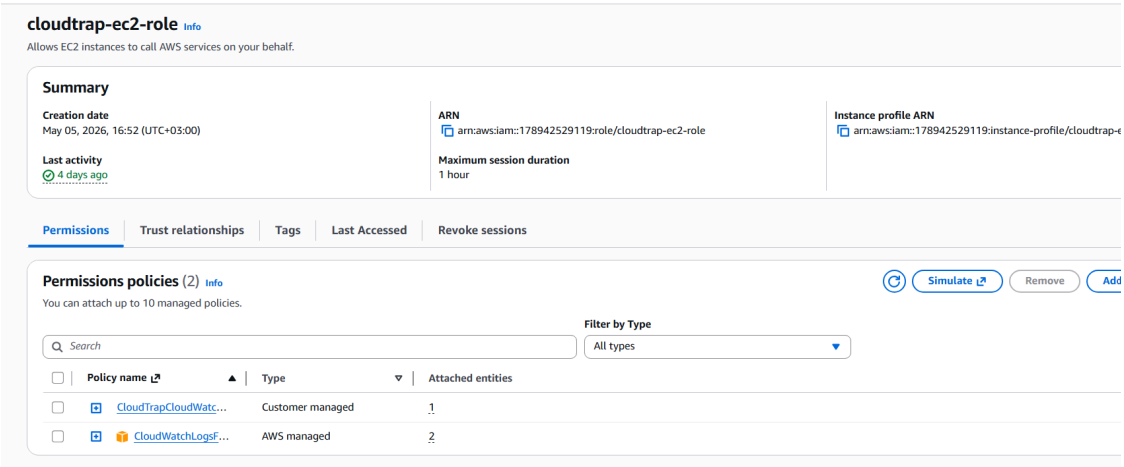
IAM Role Configuration

To enforce controlled access between the deployment and AWS infrastructure, IAM roles were used following the principle of least privilege [4], [6].



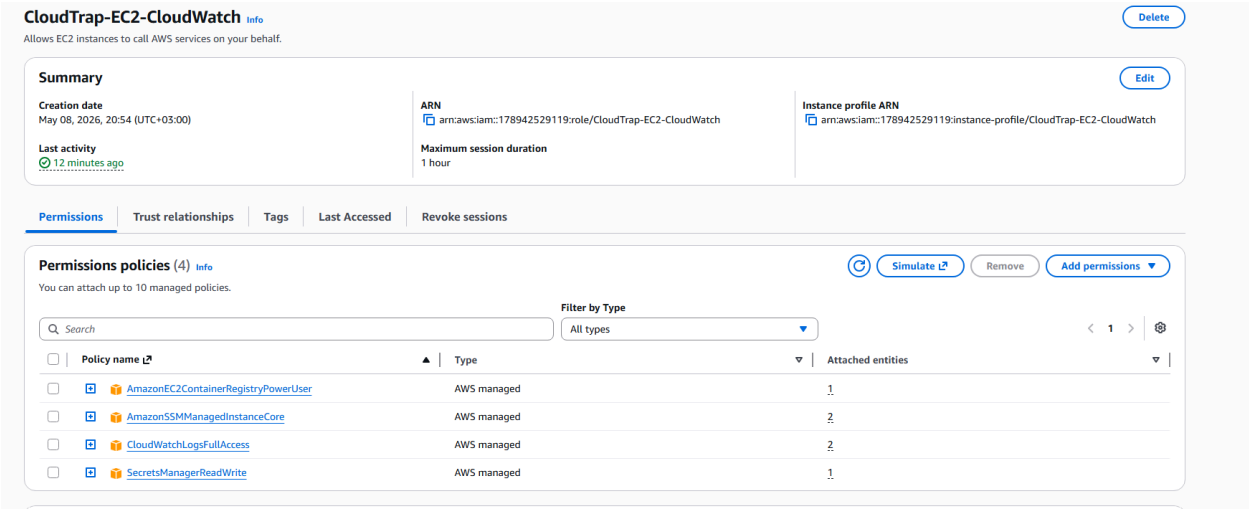
1. Cloudtrap-ec2-role

AWS CloudWatch was integrated for centralized logging, IAM was used for least-privilege access control, Secrets Manager was used to store dashboard and database credentials, ECR was used to store Docker images, and an Elastic IP was assigned to provide a stable public access point.[5]
The cloudtrap-ec2-role IAM role acted as the primary EC2 instance role attached to the CloudTrap deployment server. It provided the deployment environment with controlled permissions required for container orchestration and CloudWatch integration. The role included permissions associated with CloudWatch logging functionality through both AWS-managed and custom CloudTrap policies.



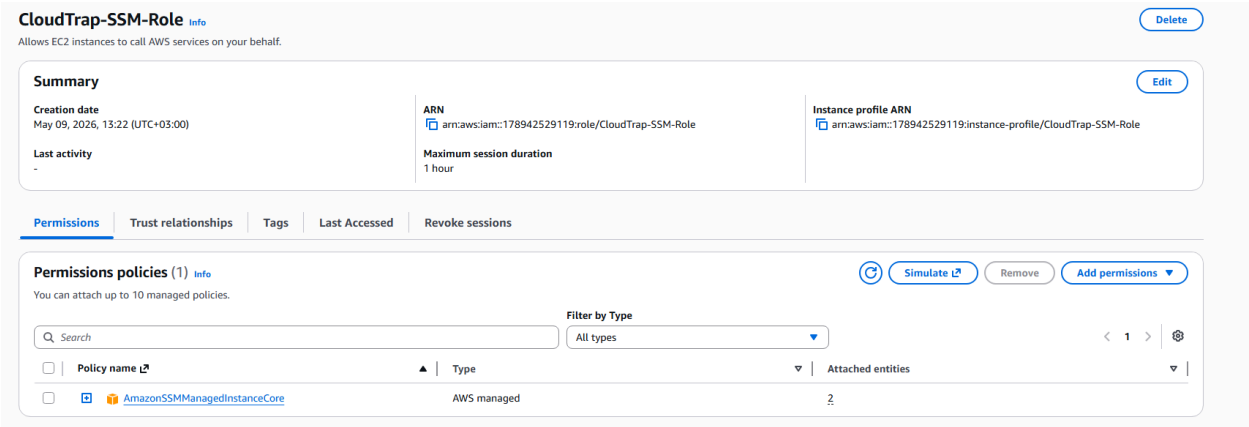
2. CloudTrap-EC2-CloudWatch

The CloudTrap-EC2-CloudWatch role was created to support integration between the deployed honeypot containers and AWS infrastructure services. This role included permissions for:
Amazon CloudWatch Logs, Amazon ECR access, AWS Systems Manager (SSM), AWS Secrets Manager.
These permissions allowed the CloudTrap deployment to:
push and retrieve centralized logs, pull container images from Amazon ECR, securely retrieve runtime secrets, and support remote instance management through AWS Systems Manager.
The role relied entirely on IAM-based authentication instead of static AWS access keys.



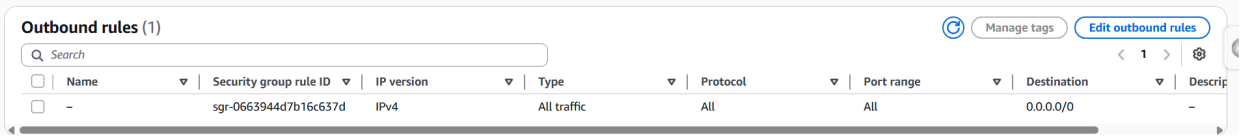
3. CloudTrap-SSM-Role

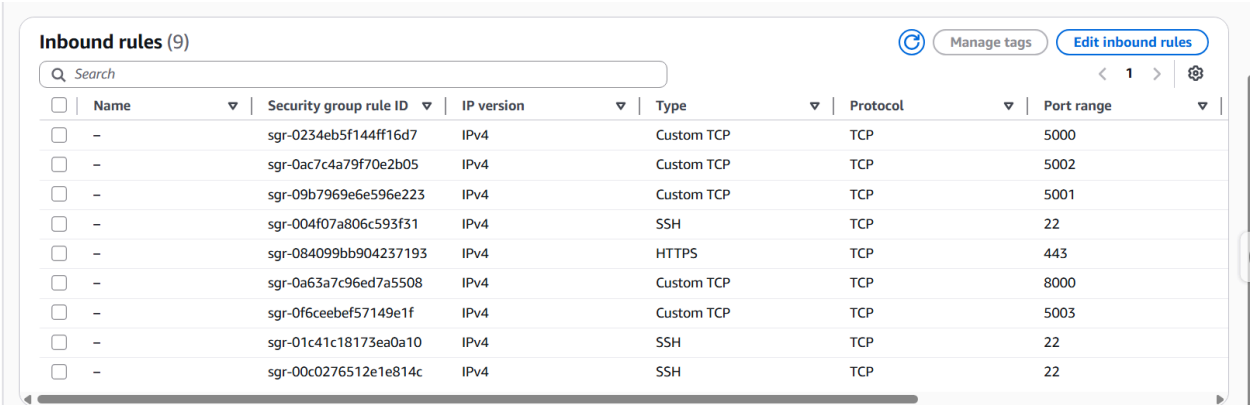
The CloudTrap-SSM-Role was configured specifically for AWS Systems Manager integration. It included the AmazonSSMManagedInstanceCore policy, allowing secure remote management of the deployment environment without exposing SSH credentials directly through the application architecture.



Security Groups

To control inbound and outbound network access AWS security groups were configured on the EC2 instance. Public access was allowed for the ports required for the honeypot services and dashboard. HTTPS traffic was also labeled during deployment and testing through port 443.
SSH administrative access was restricted to a specific trusted IP address instead of being exposed publicly to all internet sources. This reduced unauthorized access to the EC2 instance.
Outbound traffic remained enabled to allow the deployed environment to communicate with the cloud infrastructure services.[4]

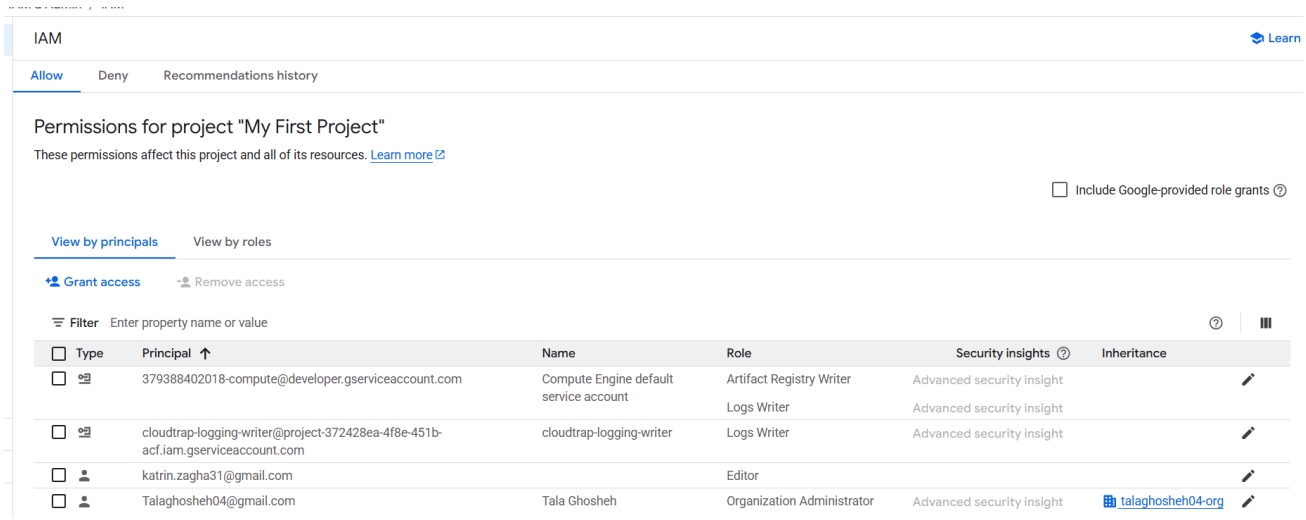




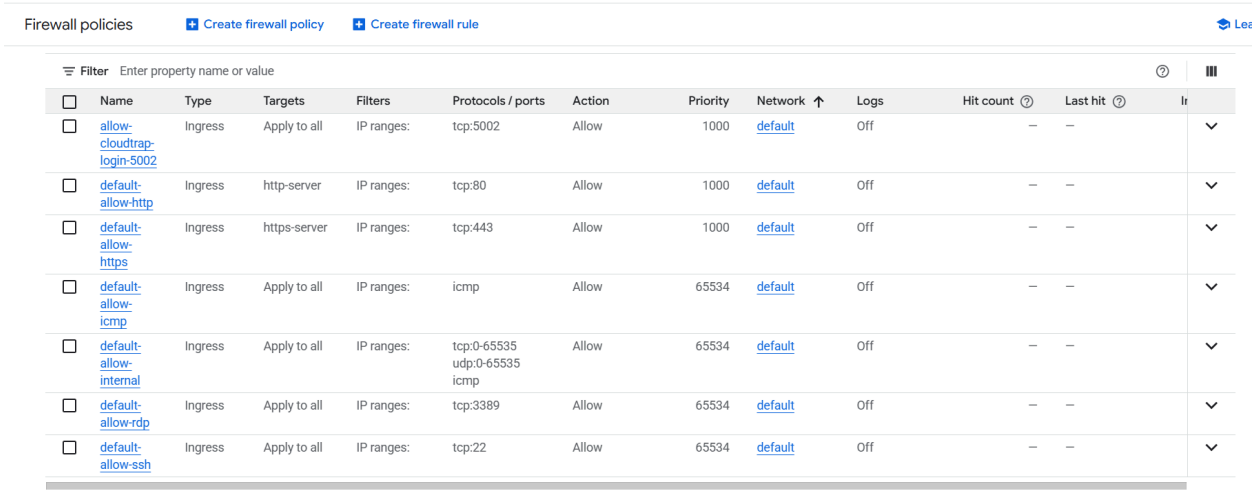
On GCP, the login portal deployment was prepared using **Compute Engine VM instance.** This supports the multi-cloud goal of the project by extending the deception network beyond a single cloud provider. Similar to the AWS deployment, Docker was used to containerize the application to maintain deployment consistency across cloud environments.
Google Cloud IAM roles and service accounts were also configured to control access between the GCP-hosted components and cloud infrastructure services.

The login portal container image was stored and managed through Google Artifact Registry before deployment onto the Compute Engine instance. This allowed centralized image management, controlled redeployment, and version tracking for the GCP-hosted deception service.
The default Compute Engine service account was granted permissions including Artifact Registry Writer and Logs Writer, allowing the deployment environment to interact with Google Artifact Registry and Cloud Logging securely during runtime.

A dedicated service account named cloudtrap-logging-writer was also configured with Logs Writer permissions to support structured logging integration for the GCP-hosted login portal service. This allowed attacker events generated on the GCP deployment to be forwarded into Google Cloud Logging without exposing long-term credentials directly inside the application code.



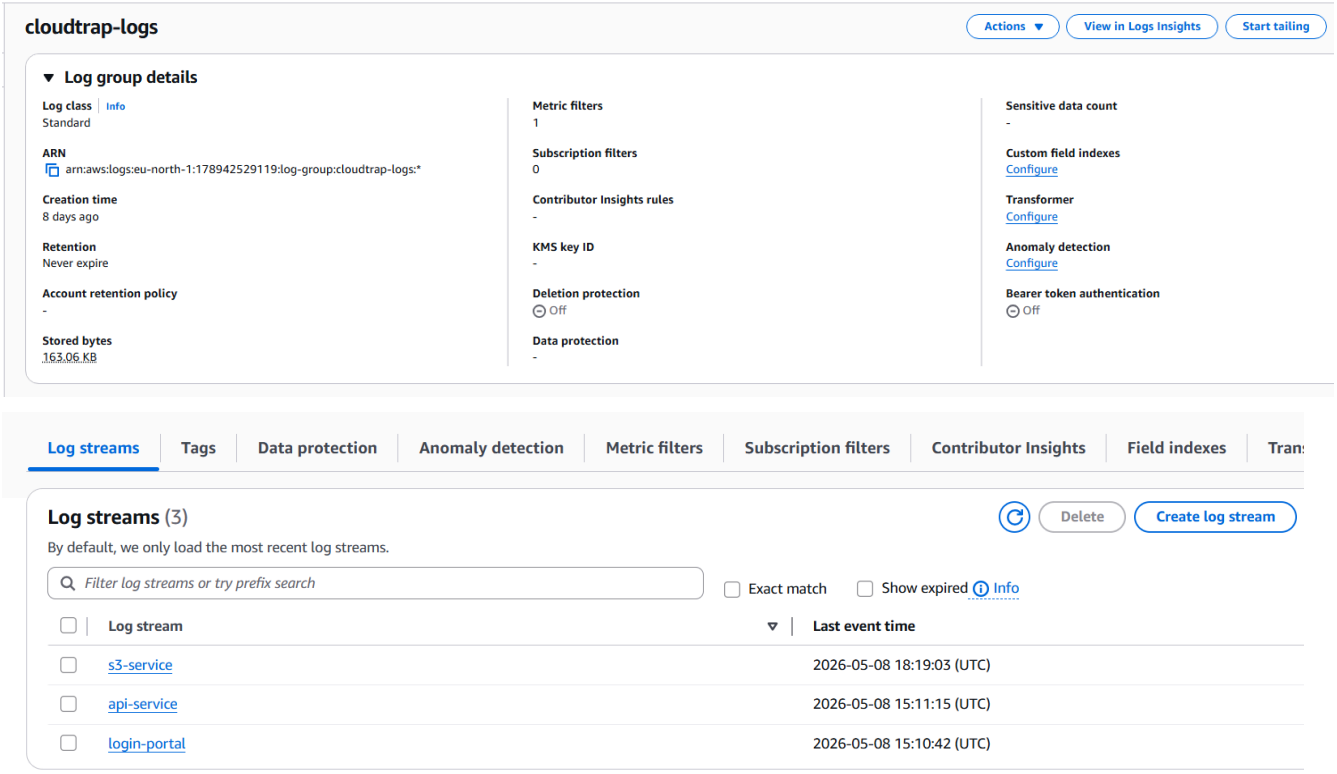
Firewall rules were configured to expose only the required network ports associated with the login portal service while maintaining controlled administrative access to the VM instance. A dedicated firewall called allow-cloudtrap-login-5002 was created, allowing the fake login portal to remain publicly reachable for attacker interactions.



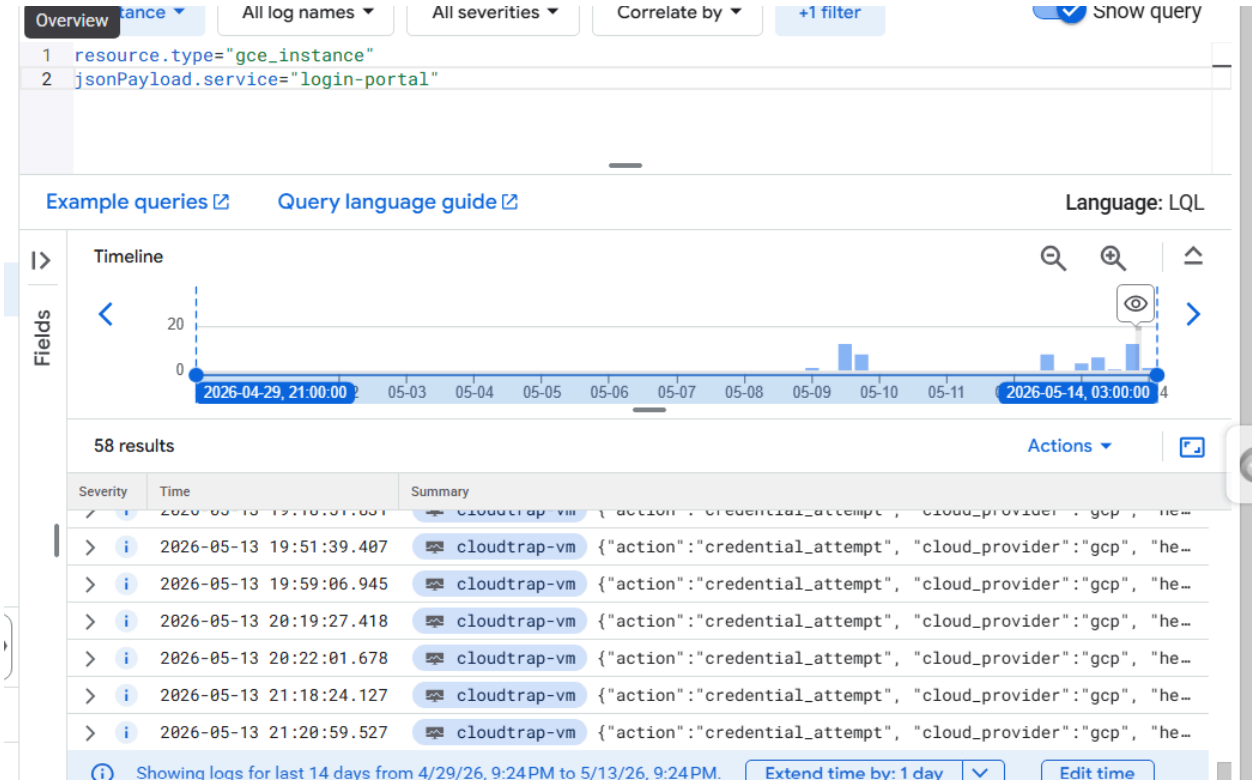
3.9 Logging and Cloud Integration

CloudTrap uses structured logging as the foundation of its monitoring pipeline. Locally, each honeypot writes events to JSONL log files. In the AWS deployment, the honeypot services also forward logs to Amazon CloudWatch Logs using the configured IAM role and boto3 integration.

A CloudWatch log group named cloudtrap-logs was created, with separate log streams for the API service, S3 service, and login portal. This separation makes it easier to monitor each honeypot independently while still allowing centralized processing.



To support the multi-cloud deployment architecture, Google Cloud Logging was also integrated for the GCP-hosted login portal service. Structured attacker events generated on the GCP deployment could therefore be monitored independently while remaining compatible with the centralized analysis workflow.

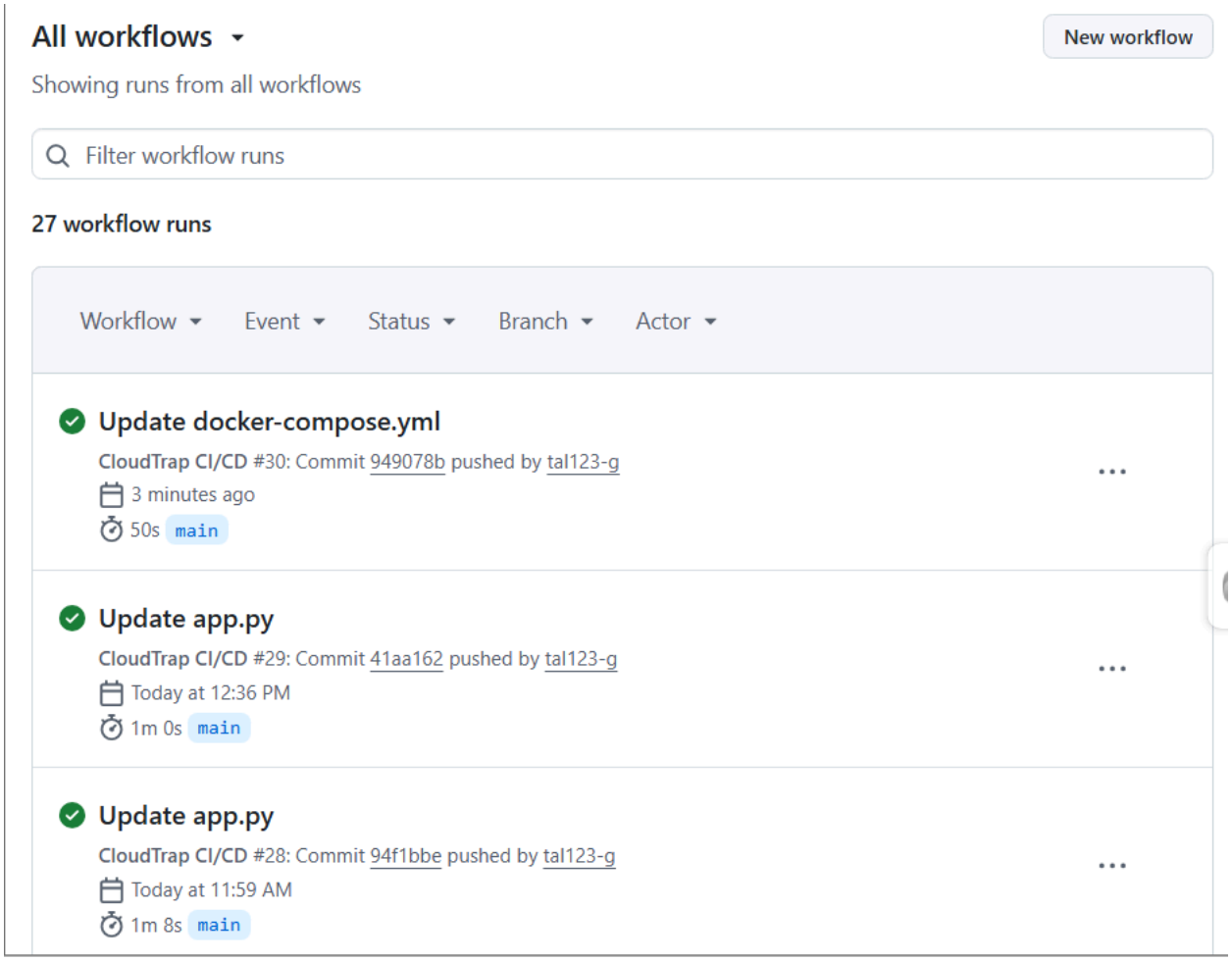


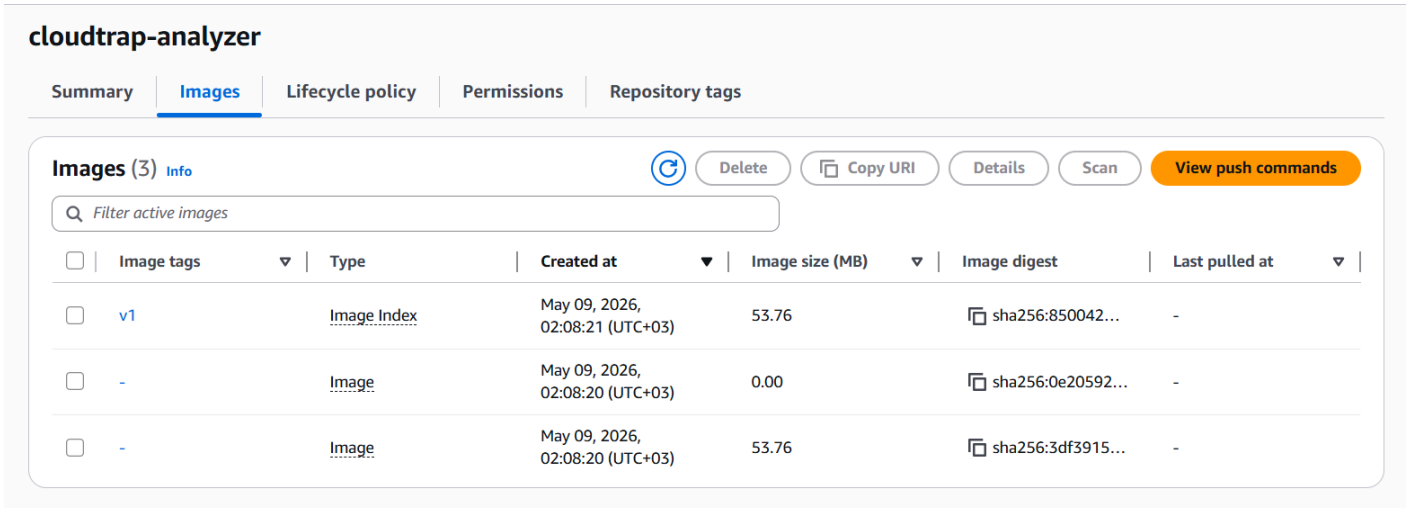
A dedicated log-forwarder service was added to bridge cloud logging with the analyzer pipeline. The forwarder polls CloudWatch Logs periodically, normalizes received log events, and writes them into the shared JSONL format consumed by the analyzer. This design allows the analyzer to work with a consistent input format regardless of where the original event was generated.

3.10 CI/CD Pipeline

A GitHub Actions CI/CD pipeline was implemented to validate the project automatically whenever changes are pushed to the repository. The pipeline performs Python syntax checks, Docker Compose validation, Docker image builds, Gitleaks secret scanning, and Trivy vulnerability scanning.

This pipeline improves the security and reliability of CloudTrap by reducing the chance of deploying broken code, leaked credentials, or vulnerable containers [8]. The pipeline was confirmed to be passing successfully after the final fixes were applied.





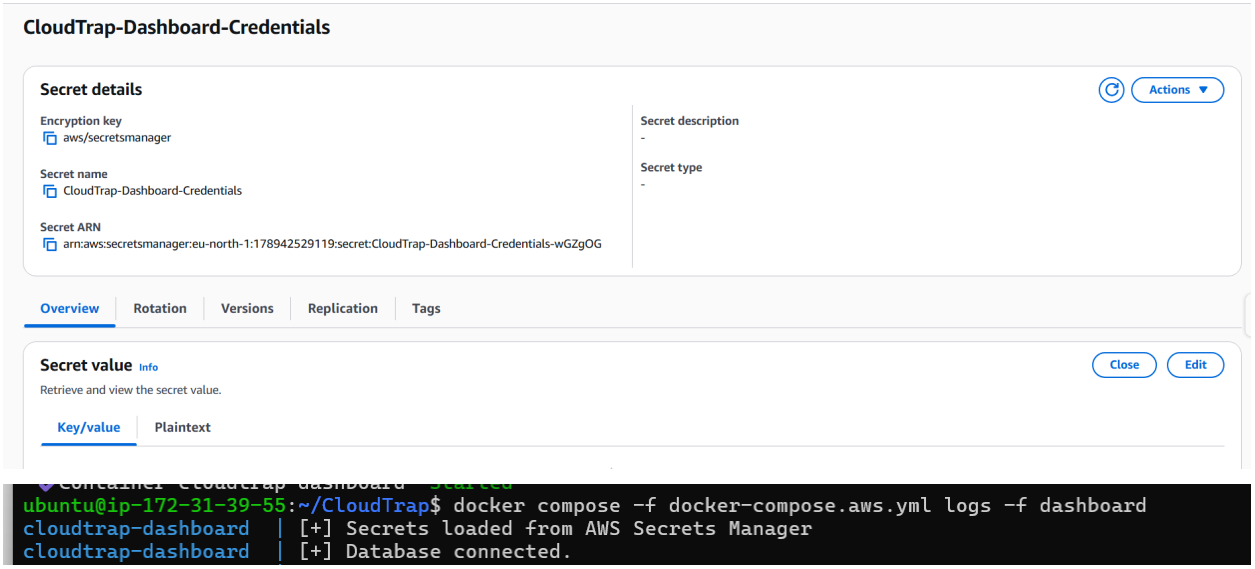
Docker images stored in Amazon ECR after successful pipeline build

4. Security Analysis

CloudTrap was designed with multiple defensive controls intended to reduce unnecessary exposure while still allowing the honeypot services to remain publicly reachable for attacker interaction. Since the project intentionally exposes decoy services to the internet, security controls focused primarily on isolation, access restriction, credential protection, and controlled monitoring rather than traditional service hardening.

IAM-based access control was implemented across the AWS deployment using dedicated EC2 roles and managed policies, as seen in Section 3.8. Instead of embedding static cloud credentials inside the application code, AWS services are authenticated through attached IAM roles. This reduced the risk of credential leakage and simplified permission management across CloudWatch, ECR, Secrets Manager, and Systems Manager integrations [6]. On the GCP side, service accounts and IAM permissions were configured to allow controlled interaction with Artifact Registry and Cloud Logging services without storing long-term credentials directly inside containers.

Sensitive configuration values, including dashboard credentials and database secrets, were stored using AWS Secrets Manager rather than being hardcoded within source files or Docker images. The dashboard dynamically retrieved these secrets during runtime. This reduced the likelihood of accidental credential exposure through source control or container image distribution.



Containerization also improved isolation between services. Each component operated within its own Docker container, reducing direct dependency conflicts and limiting the impact of service-level failures. Docker Compose health checks and automatic restart policies improved deployment reliability during runtime failures or container crashes.

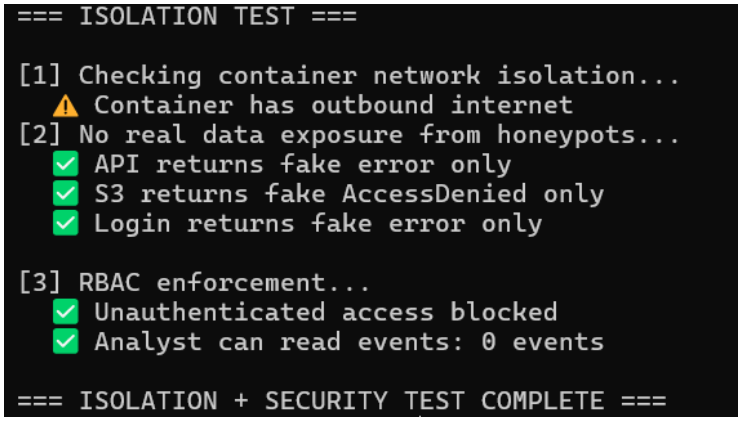
Network-level exposure was controlled using cloud firewall configurations and AWS security groups. Only the ports required for honeypot interaction and dashboard access were publicly exposed.

Administrative SSH access to the AWS EC2 instance was restricted to trusted IP addresses rather than being fully open to the internet.

Structured logging also contributed to the security model of the system. Since all attacker interactions were normalized into a unified JSON schema, the analyzer could process events consistently without directly executing or trusting attacker-supplied input. The dashboard displayed processed data retrieved from PostgreSQL instead of reading raw requests directly from the honeypot services.

Several security-oriented tests were performed during deployment. These included verifying IAM-based access to CloudWatch and Secrets Manager, validating Docker container restart behavior after simulated failures, testing structured log generation across both AWS and GCP deployments, and performing external interaction tests using browser requests and curl-based probes against the exposed honeypot services. During deployment, the system successfully captured real unsolicited reconnaissance scans, credential attempts, and path enumeration activity originating from external IP addresses.

4.1 Isolation



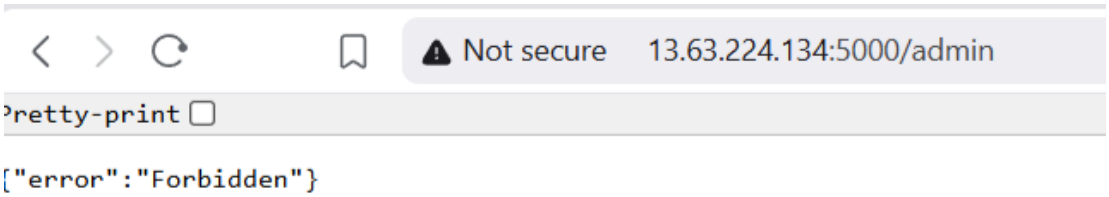
5. Evaluation & Testing

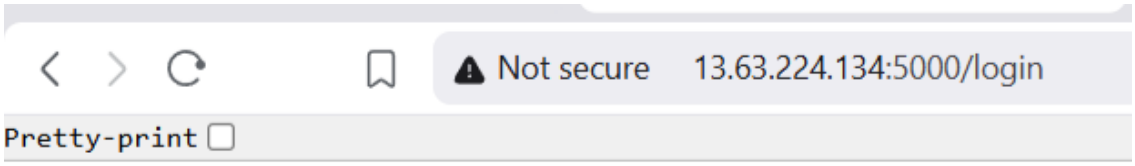
5.1 Honeypot Interaction Testing

Each honeypot service was tested manually using browser requests and curl-based HTTP requests to simulate common attacker behavior patterns.

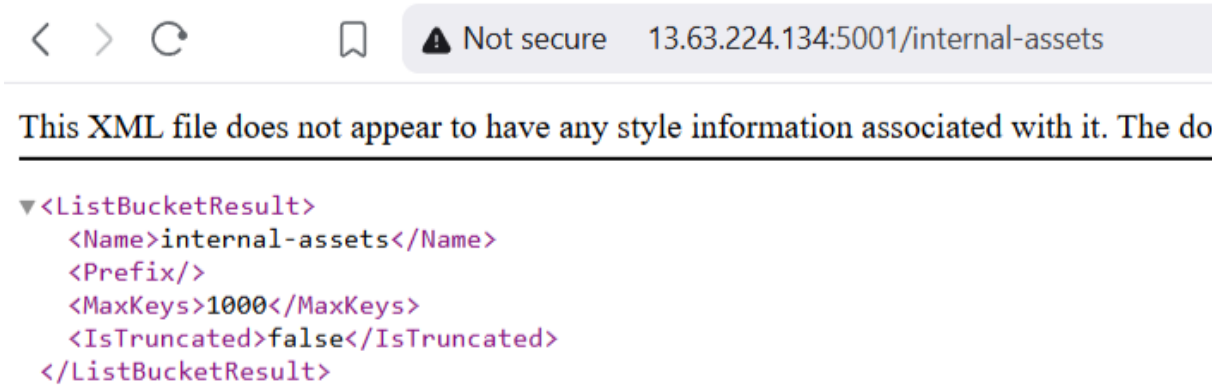
The fake REST API was tested using requests targeting sensitive endpoints such as /admin, /login, and generic API paths. Different HTTP methods, including GET and POST requests were used to verify that attacker metadata was captured correctly and normalized into structured JSON logs.
Can be accessed through: http://13.63.224.134:5000



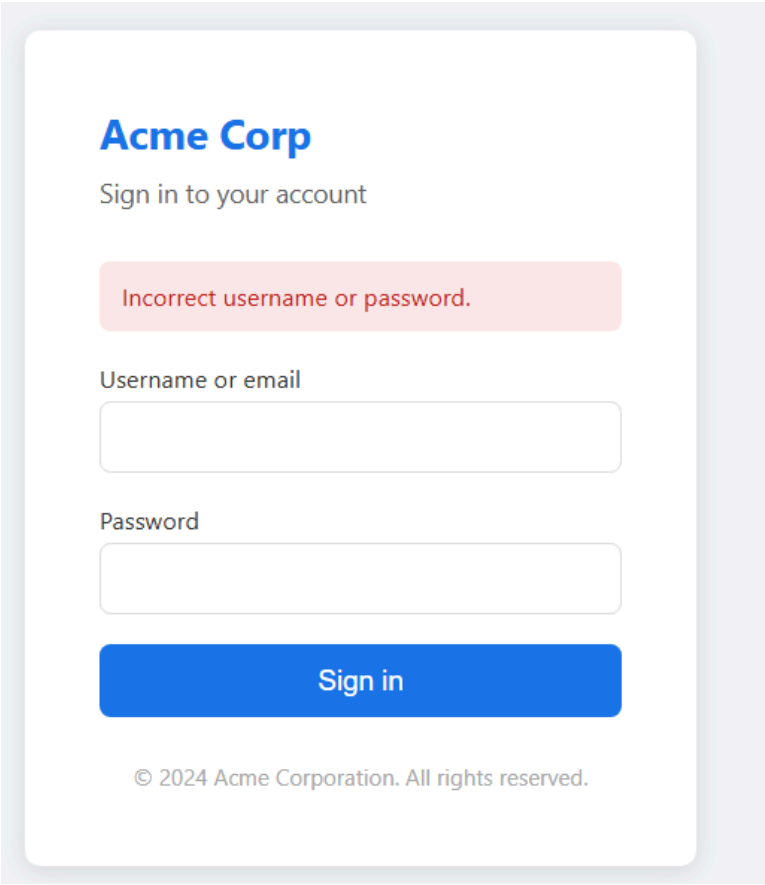




The fake S3 service was tested using bucket enumeration attempts, object-access requests, and simulated upload behavior. Requests targeting predefined fake bucket names successfully generated structured attack events categorized as bucket enumeration or object-access attempts.
Accessed thoroughly: http://13.63.224.134:5000



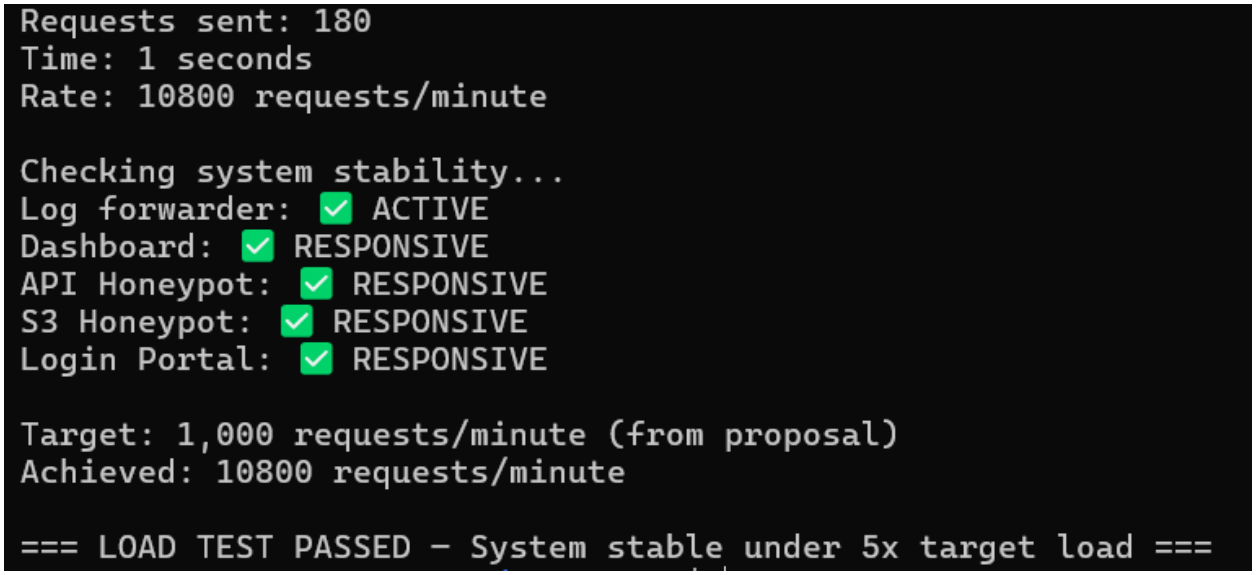
The fake login portal was tested by submitting simulated credentials through both browser interaction and command-line requests. Credential attempts were successfully logged and classified while the service consistently returned generic authentication failure responses.



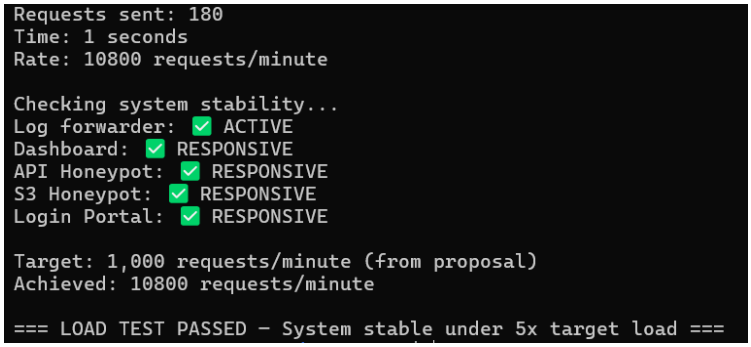
5.2 Functional Testing

```
=== FUNCTIONAL TEST ===
[1] API (5000):
  ✅ PASS
[2] S3 (5001):
  ✅ PASS
[3] Login Portal (5002):
  ✅ PASS
[4] Dashboard (5003):
  ✅ PASS
[5] Dashboard API with auth:
  ✅ PASS (37411 total events)
[6] RBAC Admin:
  ✅ PASS (role: admin)
[7] RBAC Analyst:
  ✅ PASS (role: analyst)
[8] Log Forwarder:
  ✅ PASS
```

5.3 Load Testing



5.4 Performance Testing

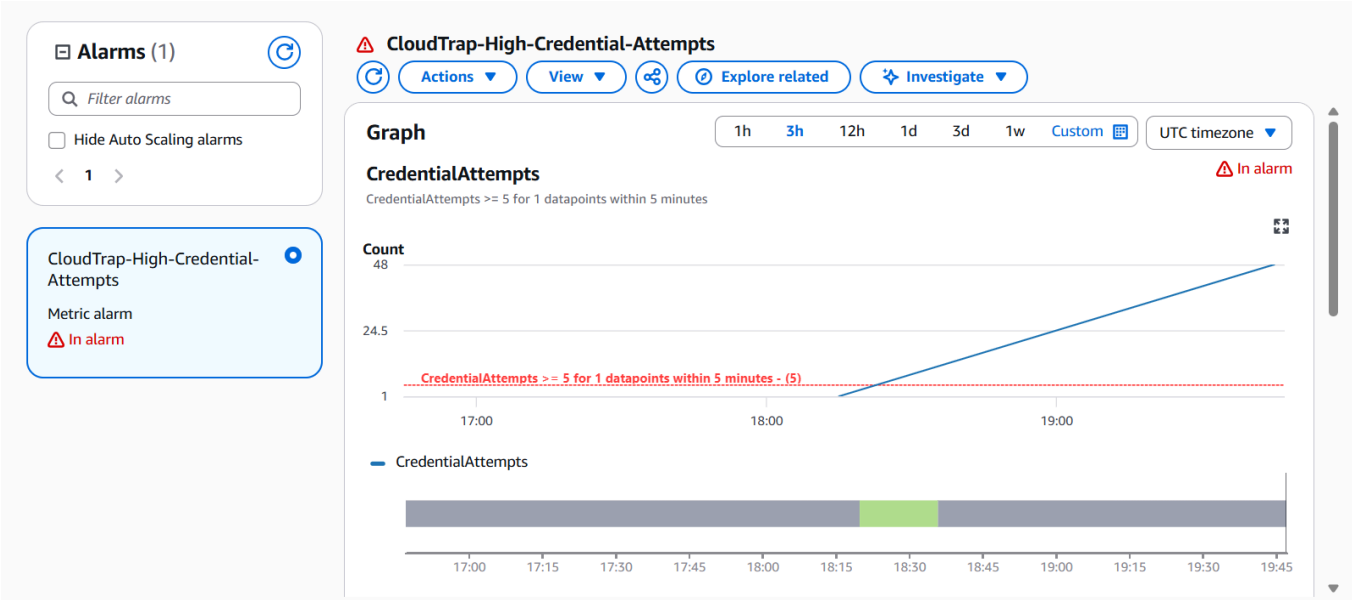


5.5 Security Testing

```
ubuntu@ip-172-31-39-55:~/CloudTrap$ echo "=== MALFORMED PAYLOAD TEST ==="
curl -s -X POST http://localhost:5000/admin -d "not json" && echo " ✅ API handles bad payload"
curl -s -X POST http://localhost:5002/login -d "" && echo " ✅ Login handles empty body"
curl -s http://localhost:5001/../../../etc/passwd && echo " ✅ S3 handles path traversal"
echo "=== ALL SERVICES SURVIVED ==="
=== MALFORMED PAYLOAD TEST ===
{"error":"Forbidden"}
 ✅ API handles bad payload
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acme Corp — Sign In</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
      background: #f0f2f5;
      display: flex;
      align-items: center;
      justify-content: center;
      min-height: 100vh;
    }
    .card {
      background: #fff;
      border-radius: 8px;
      box-shadow: 0 2px 12px rgba(0,0,0,0.1);
      padding: 40px;
      width: 360px;
    }
    .logo { font-size: 22px; font-weight: 700; color: #1a73e8; margin-bottom: 8px; }
    .subtitle { font-size: 14px; color: #666; margin-bottom: 28px; }
    label { font-size: 13px; color: #333; display: block; margin-bottom: 4px; }
    input {
      width: 100%;
      padding: 10px 12px;
      border: 1px solid #ddd;
      border-radius: 6px;
      font-size: 14px;
      margin-bottom: 16px;
```

```
      outline: none;
      transition: border-color .2s;
    }
    input:focus { border-color: #1a73e8; }
    button {
      width: 100%;
      padding: 11px;
      background: #1a73e8;
      color: #fff;
      border: none;
      border-radius: 6px;
      font-size: 15px;
      font-weight: 500;
      cursor: pointer;
    }
    button:hover { background: #1558b0; }
    .error {
      background: #fce8e6;
      color: #c5221f;
      border-radius: 6px;
      padding: 10px 12px;
      font-size: 13px;
      margin-bottom: 16px;
      display: block;
    }
    .footer { font-size: 12px; color: #aaa; text-align: center; margin-top: 24px; }
  </style>
</head>
<body>
  <div class="card">
    <div class="logo">Acme Corp</div>
    <div class="subtitle">Sign in to your account</div>
    <div class="error" id="err">Incorrect username or password.</div>
    <form method="POST" action="/login">
      <label for="username">Username or email</label>
      <input type="text" id="username" name="username" autocomplete="username" required>
      <label for="password">Password</label>
      <input type="password" id="password" name="password" autocomplete="current-password" required>
      <button type="submit">Sign in</button>
```

```
  <div class="card">
    <div class="logo">Acme Corp</div>
    <div class="subtitle">Sign in to your account</div>
    <div class="error" id="err">Incorrect username or password.</div>
    <form method="POST" action="/login">
      <label for="username">Username or email</label>
      <input type="text" id="username" name="username" autocomplete="username" required>
      <label for="password">Password</label>
      <input type="password" id="password" name="password" autocomplete="current-password" required>
      <button type="submit">Sign in</button>
    </form>
    <div class="footer">© 2024 Acme Corporation. All rights reserved.</div>
  </div>
</body>
</html> ✅ Login handles empty body
<?xml version="1.0" encoding="UTF-8"?><Error><Code>AccessDenied</Code><Message>Access Denied</Message><RequestId>CLOUDT
✅ S3 handles path traversal
=== ALL SERVICES SURVIVED ===
```



CloudWatch alarm triggered when credential attempts exceeded threshold

5.2 Centralized Logging Validation

Structured logging behavior was verified across both cloud providers. On AWS, attacker events generated by the API and S3 honeypots were successfully forwarded into Amazon CloudWatch Logs using IAM-based authentication. Separate CloudWatch log streams were created for each honeypot service, allowing centralized monitoring and log separation.

On GCP, structured attacker events generated by the login portal were successfully forwarded into Google Cloud Logging using service-account-based authentication. This confirmed that the multi-cloud logging architecture was functioning correctly across both cloud providers.
The log-forwarder service was also tested to confirm that centralized log events could be normalized and written into the shared JSONL format consumed by the analyzer.

6. Challenges & Lessons Learned

The biggest challenge encountered during the implementation of CloudTrap was integrating logging and deployment behavior consistently across both AWS and GCP environments. Although both cloud providers offer similar infrastructure capabilities, differences in IAM configuration, container authentication, logging APIs, and deployment workflows introduced additional complexity during implementation.

One of the main technical challenges involved forwarding logs generated by the GCP-hosted login portal into Google Cloud Logging while maintaining compatibility with the centralized analysis pipeline. Unlike the AWS deployment, which relied on IAM-based CloudWatch integration through EC2 instance roles,

the GCP deployment required service-account-based authentication and additional logging configuration inside the containerized environment. Several deployment issues related to container rebuilding, environment-variable synchronization, and cloud authentication behavior had to be resolved before structured logs appeared successfully inside Google Cloud Logging.

Another challenge involved managing container orchestration and deployment consistency across environments. Since multiple services depended on shared logging volumes, databases, and internal networking, even small configuration conflicts inside Docker Compose could temporarily break communication between components. This highlighted the importance of careful container orchestration, environment isolation, and configuration management within cloud-native applications.

The project also demonstrated the practical importance of centralized logging and infrastructure monitoring. During live deployment, the honeypot services began receiving unsolicited external traffic almost immediately after becoming publicly reachable. This provided direct visibility into how automated scanners and opportunistic attackers interact with exposed cloud services in real environments.[12]

From a security perspective, the implementation reinforced the importance of least-privilege IAM configuration, secret management, and avoiding hardcoded credentials inside cloud-hosted applications. It also demonstrated how lightweight deception systems can generate meaningful security intelligence without exposing legitimate infrastructure or sensitive organizational assets.

Future improvements could include machine-learning-based attack classification, automated threat scoring, long-term event correlation, support for additional cloud providers, and more advanced deception services capable of simulating realistic cloud workloads and attacker interaction patterns.[13]

7. Conclusion

CloudTrap successfully demonstrated the design and implementation of a lightweight multi-cloud honeypot architecture deployed across AWS and GCP environments. The system combined deception services, centralized logging, attack analysis, container orchestration, and dashboard visualization within a unified cloud-native security platform.

The project achieved its primary objectives by implementing realistic cloud deception services capable of safely capturing attacker interactions without exposing legitimate assets. Structured logging, centralized monitoring, and automated analysis allowed the system to transform raw attacker activity into meaningful security intelligence that could be visualized through a centralized dashboard interface.

The deployment also demonstrated the practical value of combining DevSecOps practices with deception technology. Docker containerization, IAM-based access control, cloud-native logging integrations, CI/CD automation, and secret management collectively improved deployment consistency, monitoring visibility, and operational security throughout the project.

During deployment, CloudTrap successfully captured thousands of real attacker-generated events, including credential attempts, reconnaissance scans, administrative probing, and object-storage enumeration activity originating from external IP addresses. These results demonstrated that publicly exposed cloud services are continuously targeted by automated scanning activity and opportunistic attackers shortly after deployment.

Overall, CloudTrap shows that a practical and scalable multi-cloud deception platform can be implemented using modern cloud infrastructure and lightweight open-source technologies while still providing valuable threat visibility and centralized attacker intelligence.

8. References

[1] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2002.

[2] N. Provos and T. Holz, *Virtual Honeypots: From Botnet Tracking to Intrusion Detection*. Addison-Wesley Professional, 2007.

[3] M. Nawrocki, M. Wählisch, T. C. Schmidt, C. Keil, and J. Schoenfelder, "A Survey on Honeypot Software and Data Analysis," *arXiv:1608.06249*, 2016.

[4] Amazon Web Services, "AWS Well-Architected Framework – Security Pillar," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html

[5] Amazon Web Services, "Amazon CloudWatch User Guide," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/

[6] Amazon Web Services, "AWS Identity and Access Management User Guide," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/IAM/latest/UserGuide/

[7] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux Journal*, vol. 2014, no. 239, 2014.

[8] Docker Inc., "Docker Compose Documentation," Docker Documentation, 2024. [Online]. Available: https://docs.docker.com/compose/

[9] MITRE, "ATT&CK for Cloud," MITRE Corporation, 2024. [Online]. Available: https://attack.mitre.org/matrices/enterprise/cloud/

[10] CSA, "Top Threats to Cloud Computing: Pandemic Eleven," Cloud Security Alliance, 2022. [Online]. Available: https://cloudsecurityalliance.org/

[11] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/

[12] R. Baykara and Z. Z. Gürel, "A Survey on Potential Applications of Honeypot Technology in Intrusion Detection Systems," *International Journal of Computer Networks and Applications*, vol. 6, no. 1, pp. 25–36, 2019.

[13] A. Vetterl and R. Clayton, "Honware: A Virtual Honeypot Framework for Capturing CPE and IoT Traffic," in *Proc. APWG Symp. on Electronic Crime Research (eCrime)*, 2019, pp. 1–13.

[14] T. Ristenpart, E. Tromer, H. Shacham, and S. Savage, "Hey, You, Get Off of My Cloud: Exploring Information Leakage in Third-Party Compute Clouds," in *Proc. ACM CCS*, 2009, pp. 199–212.

[15] D. Dittrich and E. Kenneally, "The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research," U.S. Department of Homeland Security, 2012.